TopoGraph: Batched GPU Simulation for Graph-Structured Reinforcement-Learning Environments with Variable Topology

Danny Mottesi · mottesid@stanford.edu
CS348K: Visual Computing Systems — Final Project, Spring 2026

Abstract — Batched GPU simulators accelerate reinforcement learning by orders of magnitude, but they assume a fixed environment topology. This work investigates whether this GPU-resident paradigm extends to environments where each parallel instance dynamically constructs a different graph. Our workload is transit-network construction on synthetic grid cities. The technical crux is all-pairs shortest paths (APSP) across a batch of graphs with different active edge sets, which is ~86% of per-step cost. We resolve it with edge-existence masking plus fixed-iteration min-plus matrix-squaring, which is bit-exact with Floyd–Warshall at K=5 on our grids. On an NVIDIA A100 the full simulator reaches 60× the CPU baseline at batch 256; variable topology costs ≤1.2% over a fixed-topology baseline; and end-to-end training (policy forward + REINFORCE backward) retains 94–98% of simulator throughput because the policy and simulator stay on-device under one compiled program. A where-the-time-goes analysis shows the speedup comes from batching and fusing the memory-bandwidth-bound APSP across environments — the bottleneck does not move, it intensifies.

1 Background and Setup

Modern reinforcement learning is heavily bottlenecked by environment simulation. To overcome this, recent frameworks like Madrona [1], Brax [2], and Isaac Gym [3] deliver order-of-magnitude throughput gains by running batched simulations entirely on the GPU. However, a core assumption underpins these frameworks: the environments must share a fixed topology. Whether managing the same rigid bodies, contact models, or underlying graphs, this structural uniformity is precisely what allows a single vectorized kernel to execute across the entire batch simultaneously.

Many critical RL domains violate this assumption. In this work, we study **transit-network construction**, where an agent incrementally builds a public-transit network across a city. Here, the environment state (the reachability graph) dynamically changes shape as edges are added. Consequently, after only a few steps, every parallel environment in a batch contains a distinct set of active edges, resulting in highly heterogeneous graphs. This project investigates whether the high-throughput batched-simulator paradigm can survive and adapt to such dynamic, environment-specific topologies.

Inputs, outputs, goals, and constraints

Inputs / outputs. An environment is a synthetic grid city of N zones (we target 10×10 to 15×15, i.e. N=100 to 225) with a fixed set of candidate transit edges and a per-environment edge

budget. Each of the 15 episode steps, the agent activates one candidate edge (or no-ops); the per-step *output* is a scalar reward of accessibility-weighted welfare computed over the all-pairs travel-time matrix of the current network. The quantity of interest for this project is not reward but *throughput*: environments (and environment-steps) simulated per second.

Goals / constraints. (i) fit a single GPU at the targeted grid sizes; (ii) remain bit-faithful to the CPU reference simulator's semantics, not a hand-waved approximation; (iii) no Python loop over environments anywhere on the hot path — the systems discipline that makes batching possible.

Question / hypothesis

A graph-structured RL environment with variable topology can be batched on one GPU with (RQ1) ≥10× simulator throughput over a fair CPU baseline at acceptable accuracy and within 24 GB; (RQ2) only single-digit-percent overhead versus an otherwise-identical *fixed*-topology simulator; and (RQ3) without the policy/host boundary erasing that throughput once a learner is in the loop.

The crux of the technical problem

Reward requires all-pairs shortest paths (APSP) every step, and on the CPU baseline APSP is **~86% of per-step time** (Section 3). A GPU batches efficiently when every environment runs the *same* computation over the same-shaped data, but here each environment has a different active edge set, so a fixed shortest-path kernel does not obviously apply, and the textbook APSP (Floyd–Warshall) is a sequential triple loop that is poorly suited to the GPU. Making batched APSP across varying edge sets both fast and correct is the make-or-break problem.

2 Approach

What we started with. No suitable slim simulator existed, so I built the CPU reference from scratch in NumPy/SciPy. I then ported the simulator to JAX.

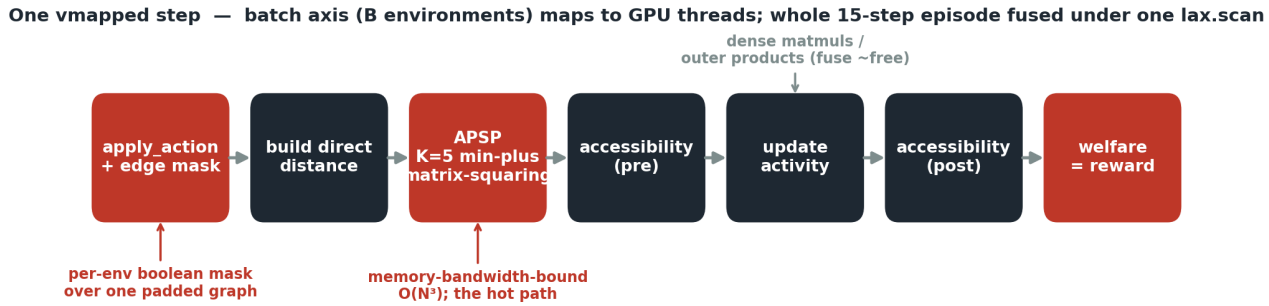

Figure 1. The per-step simulator, written once for a single environment and *vmap*'d over the batch. The whole 15-step episode is fused under one *lax.scan*. Shortest paths (red) are the hot path; the dense dynamics (dark) fuse to near-free on the GPU.

Edge-existence masking

We keep *one* padded global graph and give each environment a boolean mask over the candidate edges. A different graph is then just a different mask over identically-shaped arrays, with JIT-stable shapes, no ragged tensors, and no per-environment branching. The per-step

direct-distance matrix is the walking-time floor with active-edge travel times scattered in; inactive edges contribute +∞ and leave the floor untouched.

Batched APSP via fixed-iteration min-plus matrix-squaring

We compute APSP in the (min, +) semiring: distance update $D \leftarrow \min(D, D \oplus D)$, where $\oplus$ is min-plus matrix multiply. Squaring the distance matrix doubles the number of hops it accounts for each iteration, so it converges in $\lceil \log_2(\text{diameter}) \rceil$ iterations rather than the diameter-many a Bellman-Ford-style relaxation ($D \leftarrow \min(D, D \oplus A)$) needs. It is a dense, regular, batchable tensor operation, precisely the shape a GPU wants, in place of an irregular sequential algorithm.

Why K=5? Each squaring step doubles the hop budget, so after K steps the result is the exact shortest path among all routes of ≤ 2^K hops; it equals the true all-pairs answer once 2^K exceeds the graph's hop-diameter — i.e. $K \geq \log_2(\text{diameter})$. The walking floor keeps that diameter small: every adjacent zone pair is always walkable, so the worst case is a cell-by-cell walk across the grid — ≤ 28 hops on 15×15 — and transit edges only add shortcuts. Since $\log_2(28) \approx 4.8$, five steps suffice; empirically the output is already bit-identical to Floyd–Warshall by K=4, so we fix K=5 (one safety iteration) uniformly across all grid sizes. Because the hop budget exceeds the diameter, the result is the exact APSP rather than a bounded approximation — verified bit-exact against scipy Floyd–Warshall (Section 4.6) — so K is fixed by graph diameter, not a knob traded against accuracy, giving a constant O(log D) cost per step.

GPU-resident state and how it maps to the hardware

State is laid out ECS-style (à la Madrona [1]): a NamedTuple of per-attribute arrays with the batch axis leading (edge_mask (B,E), activity (B,N), budget (B,), …), sized to a static maximum. The single-environment step is written once and *vmap*'d; the *B* environments map to GPU threads/SMs, and the whole 15-step episode is fused into one *lax.scan* under a single dispatch. Within a step, the min-plus matmul is the work that saturates the device: it is a memory-bandwidth-bound reduction over an (N×N×N) contraction, batched to (B×N×N×N). The dense dynamics (gravity demand, exp-decay accessibility, land-use update) are ordinary matmuls and outer products that XLA fuses and that cost almost nothing once parallelized.

For RQ3: an on-device learner

To measure training throughput, we use a PureJaxRL-style [4] loop: a small (64-unit) MLP policy and a REINFORCE update live under the *same* jit/scan as the simulator, so nothing crosses to the host mid-rollout. The agent is deliberately not trained to compete; it exists to consume the simulator at a realistic rate so the boundary cost is real.

What we tried that did not work (and what we learned)

- **Hand-vectorized NumPy Floyd–Warshall as the CPU baseline.** It ran at 66 rollouts/sec; scipy's C Floyd–Warshall hit 117.8 with bit-identical output. Quoting the slow one would have inflated every GPU speedup by 1.8×, so I adopted scipy as the honest baseline and kept the NumPy version only as a GPU-port correctness reference.
- **Bellman-Ford-style relaxation.** Same answer as matrix-squaring but ~4× the iterations (K≈16 vs 4 on the larger grids). I switched to squaring.

- **Fearing an O(B·N³) memory wall.** The (B,N,N,N) min-plus intermediate is 11.7 GB at 15×15/256 on paper, and I budgeted bf16 / K-axis tiling for it. It never materialized: XLA fuses the broadcast-add-and-min into one reduce kernel, so peak memory is O(B·N²). The mitigation was unnecessary, and ended up being a cool result!
- **Measuring variable-topology overhead under a fused scan.** With the mask frozen, a fused scan lets XLA hoist the (now loop-invariant) APSP out of the 15-step loop and compute it once, reporting a bogus ~1000% "overhead." I measure per-step via a Python loop so both arms do equal APSP work and the difference is purely the mask scatter

3 Evaluation and Results

3.1 Definition of success and experimental setup

Success, in our terms, is the three research questions cleared at pre-registered gates: **RQ1** — mean travel-time error < 2% vs. exact APSP, ≥10× simulator rollouts/sec at batch 256 vs. the CPU baseline, and a fit within 24 GB at 10×10–15×15; **RQ2** — single-digit-percent overhead vs. a fixed-topology variant; **RQ3** — training throughput that tracks, rather than collapses below, simulator throughput.

Setup. CPU baseline: scipy.sparse.csgraph Floyd–Warshall on aarch64 Linux (Python 3.10, NumPy 2.2.6), measured on the reference workload — 10×10 grid, horizon 15, uniform-random-legal policy, 1000 episodes — at **117.8 rollouts/sec** (1,767 env-steps/sec; mean return 204.2 ± 4.59). GPU: NVIDIA A100-40GB via Modal (A100-80GB for the largest graph-size cells), JAX 0.5.3 / CUDA 12, fp32, K=5.

We report rollouts/sec and env-steps/sec (= batch × horizon ÷ wall-time of one on-device rollout), peak device memory, per-component step time, variable-topology overhead %, and travel-time error vs. scipy. Each cell is the median of 5 timed rollouts after 2 warm-up rollouts that absorb JIT compilation. The dataset is procedurally generated grid cities; there is no train/test split because the object of study is the simulator, not a learned policy.

3.2 RQ1 — Throughput scales with batch: 60× the CPU baseline

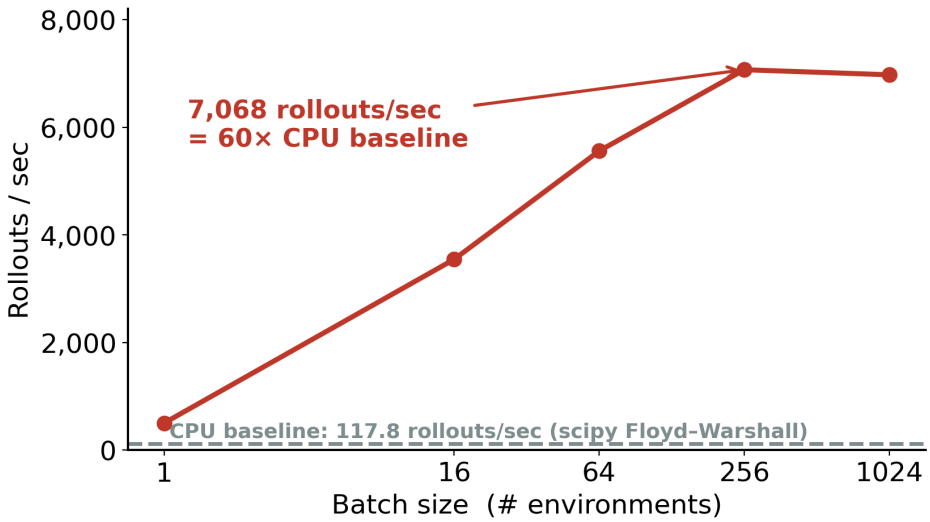

Figure 2. Full-simulator throughput vs. batch size on 10×10/horizon-15, A100. Dashed line is the scipy CPU baseline (117.8 rollouts/sec). Throughput climbs and saturates near batch 256 at 7,068 rollouts/sec = 60×.

At batch 256 the full simulator runs at **7,068 rollouts/sec — 60.0× the CPU baseline**, clearing RQ1's throughput bar by 6×. The curve rises steeply from batch 1 and flattens past 256: below 256 the GPU is under-occupied and throughput is roughly linear in batch; past 256 the device is saturated and per-rollout work, not occupancy, sets the rate. Two interpretive points. First, at batch 1 the full simulator is *slower* than the APSP kernel alone (503 vs. 727 rollouts/sec) because fixed per-rollout dispatch cost dominates; the curves cross as the batch fills the GPU. Second, and counter-intuitively, the full 15-step simulator (7,068) is ~18% *faster* than our earlier APSP-only micro-benchmark (5,994): the APSP-only version issued 15 separate kernel dispatches over a static matrix, whereas the full simulator fuses the entire episode into one lax.scan dispatch. The win is not only vectorization across environments but fusing the whole rollout into a single compiled program.

3.3 RQ3 — Training throughput tracks the simulator (94–98% retained)

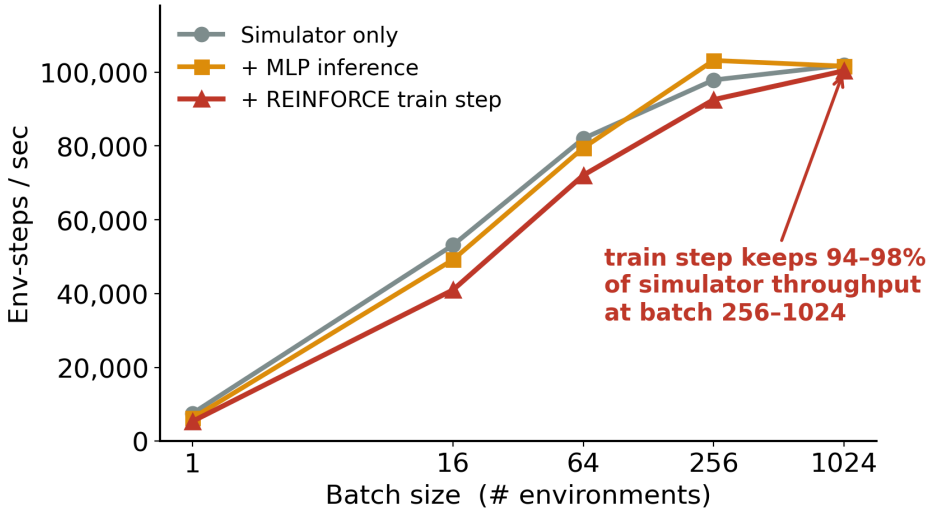

Figure 3. End-to-end throughput on 10×10, A100: simulator only (gray), + MLP inference (amber), + full REINFORCE train step (red). At batch 256–1024 the train step retains 94–98% of simulator throughput.

Next we check if the speedup survives a learner. Measuring env-steps/sec for three on-device drivers, the full training step (policy forward + REINFORCE backward + SGD update) **retains 94.5% of simulator-only throughput at batch 256 and 98% at 1024**. The gap is largest at batch 1 (71%), where fixed per-step policy cost is not yet amortized, and closes monotonically as the batch fills the GPU. The reason the host boundary never bites is structural: the policy and the simulator share one jit/scan, so there is no per-step CPU↔GPU transfer to dominate. This answers RQ3 affirmatively for the on-device regime.

3.4 Scaling with graph size: the ceiling is compute, not memory

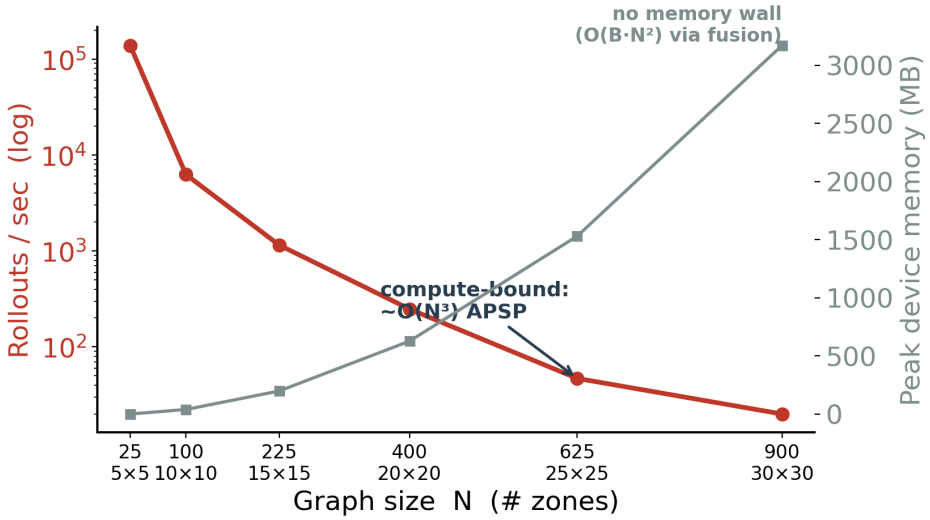

Figure 4. Throughput (red, log scale) and peak memory (gray) vs. N at batch 256, A100. Throughput falls ~O(N³); memory tracks O(B·N²) and never hits a wall.

Grid	N	rollouts/sec	peak MB
5×5	25	138,530	1
10×10	100	6,252	40
15×15	225	1,152	199
20×20	400	248	628
25×25	625	47	1,531
30×30	900	20	3,172

Two findings, and they are the opposite of what was budgeted for. **(1) The memory wall never bites.** Peak memory tracks O(B·N²), not the naive O(B·N³); even 30×30 at batch 1024 peaks at 12.7 GB, far under 40 GB, because XLA fuses the cubic intermediate away (Section 2). **(2) The ceiling is compute.** Per-rollout time scales ~O(N³) (the K=5 min-plus APSP), so throughput falls from 6,252 rollouts/sec at 10×10 to 20 at 30×30 and becomes compute-impractical long before it could OOM. It looks like this because the fused dynamics are cheap and the dense min-plus contraction dominates; the slope on the log plot is consistent with cubic growth in N.

3.5 Where the time goes — and what limits the speedup

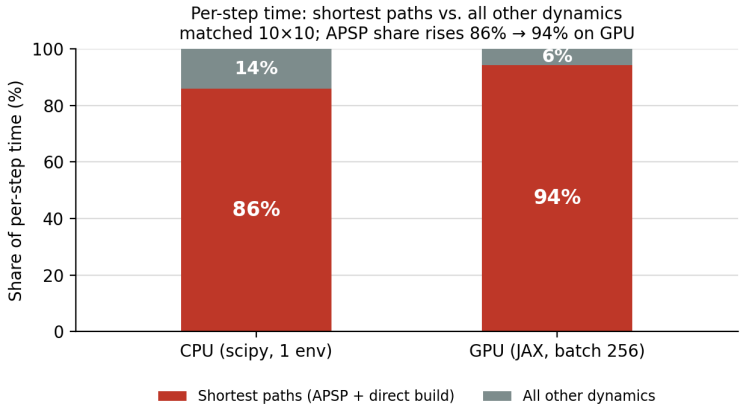

Figure 5. Per-step time split into shortest paths vs. all other dynamics, CPU (scipy) vs. GPU (batch 256), matched 10×10. The APSP share rises from 75% to ~94%.

So then what limits the speedup? On the CPU, the shortest-path block is ~86% of the step (demand ~7%, accessibility ~6%, the rest <2%). One might expect the GPU to flatten that. It does the opposite: the shortest-path share **rises to ~94%** at 10×10 (and ~97% at 15×15). The reading, from the fused whole-step ground truth (3.34 ms; APSP 2.64 ms + direct-distance build 0.51 ms = 3.15 ms ≈ 94%): the dense dynamics parallelize and fuse to near-free, while the $O(N^3)$ min-plus APSP stays **memory-bandwidth-bound** and remains the wall. So the limiter is not a lack of parallelism, nor synchronization, nor host transfer — it is the bandwidth of the min-plus contraction itself. The bottleneck intensifies, which validates the kernel as exactly the right thing to have optimized.

3.6 RQ2 — Variable topology is effectively free

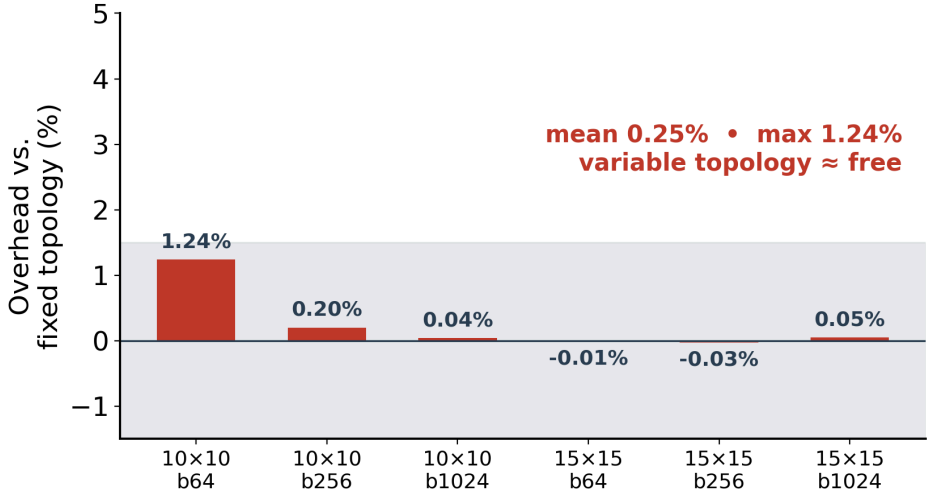

Figure 6. Overhead of live edge-mask updates vs. a frozen-topology baseline, measured per-step so both arms run all 15 APSPs. Mean 0.25%, max 1.24%.

Against an otherwise-identical fixed-topology variant (mask frozen at episode start), variable topology costs **at most 1.2% and 0.25% on average**, falling into measurement noise as batch and grid grow. The reason is immediate from Section 3.5: the only added work is the per-step edge-mask scatter in apply_action, which is negligible against the K=5 APSP that dominates the step.

3.7 Accuracy and generalization

Accuracy. Across a 630-cell sweep (3 grids × 5 mask densities × 3 seeds × 2 algorithms × 7 K values) the worst mean relative travel-time error at K=5 is **0.003%**; most cells are bit-identical to scipy Floyd–Warshall.

Generalization, and its failure case. To check the kernel is not specific to our grids, we dropped it into the unrelated transit_learning codebase [5] and benchmarked against its own Floyd–Warshall on the real Mandl and Mumford city graphs (N = 15–127). It is bit-exact at K=5 and **20–117× faster in the single-graph / small-batch regime** (it needs only $\lceil \log_2 N \rceil$ sequential steps vs. FW's N), narrowing to ~1.5–2× when hundreds of large graphs are batched under PyTorch eager — where, lacking XLA's fusion, it does hit the $O(B \cdot N^3)$ memory wall that fusion eliminated for us. The honest failure case: the “exact at K=5” guarantee relies on the walking floor bounding the hop-diameter. A sparse graph with no walking floor (e.g. a road network without a dense pedestrian fallback) would have a larger diameter, need a larger K, and could reintroduce an accuracy/throughput trade-off.

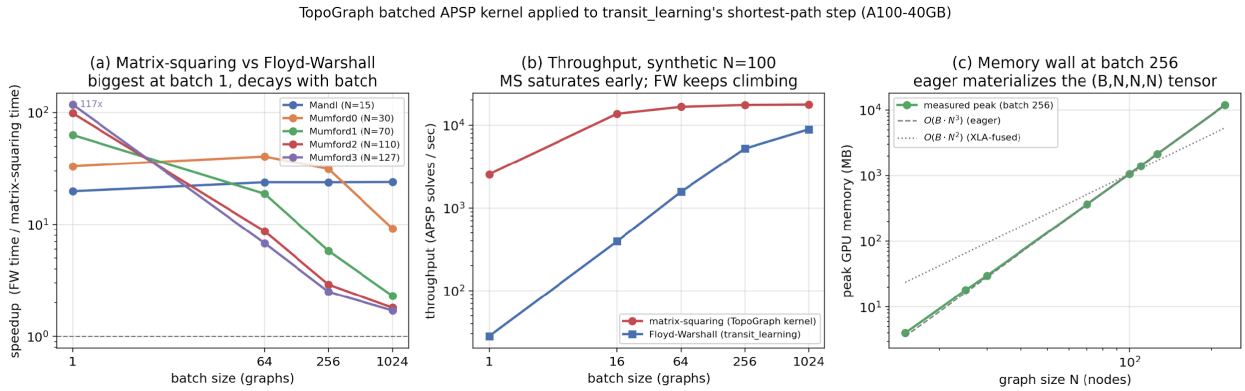

Figure 7. *TopoGraph's APSP kernel (PyTorch port) dropped into transit_learning's shortest-path step (A100-40GB). (a) speedup over transit_learning's Floyd–Warshall is largest at batch 1 (up to 117× on Mumford3) and decays as the batch grows; (b) on synthetic N=100 the kernel saturates early while FW keeps climbing; (c) under PyTorch eager the kernel materializes the (B,N,N,N) intermediate and tracks $O(B \cdot N^3)$ — the memory wall XLA fusion eliminated for TopoGraph.*

Table 1. *Matrix-squaring vs. transit_learning's Floyd–Warshall on the real Mandl/Mumford graphs (A100, fp32; speedup = FW time ÷ matrix-squaring time, median of 30 calls). Bit-exact at K=5 on every city; the single-graph win is large but narrows as batched eager hits the $O(B \cdot N^3)$ wall.*

City	N	batch=1	batch=256	batch=1024	K=5 bit-exact?
Mandl	15	19.8×	23.8×	23.9×	yes
Mumford0	30	33.1×	31.3×	9.2×	yes
Mumford1	70	63.0×	5.8×	2.3×	yes
Mumford2	110	98.4×	2.9×	1.8×	yes

City	N	batch=1	batch=256	batch=1024	K=5 bit-exact?
Mumford3	127	117.4×	2.5×	1.7×	yes

3.8 Summary of results vs. goals

Goal	Gate	Result
RQ1 throughput	≥ 10× at batch 256	60.0× (7,068 rollouts/sec) — PASS
RQ1 accuracy	< 2% travel-time error	0.003% worst; bit-exact at K=5 — PASS
RQ1 memory	fit 24 GB at 10×10–15×15	≤ 0.8 GB across sweep — PASS
RQ2 overhead	single-digit %	0.25% mean, 1.2% max — PASS
RQ3 training	throughput not erased	94–98% retained on-device – PASS

4 Discussion and Limitations

What transfers from the fixed-topology playbook: GPU-resident ECS state, vmap'd dynamics, on-device policy, and whole-rollout fusion all carry over unchanged. What does *not* transfer is the assumption that one kernel covers the batch because the structure is shared; our contribution is showing that edge-existence masking plus a fixed-iteration min-plus APSP recovers that property for variable-topology graphs at ≤1.2% cost. Limitations we are explicit about: (i) the ceiling is the $O(N^3)$ APSP, so very large graphs are compute-impractical well before any memory limit; (ii) the exactness result depends on the walking floor and would weaken on graphs without one; (iii) we characterize fp32 only; (iv) RQ3 is answered on-device — a multi-process CPU training baseline (sketched, not run) is the remaining piece for a full GPU-vs-CPU training speedup; (v) single-GPU only.

References

[1] B. Shacklett, L. G. Rosenzweig, Z. Xie, B. Sarkar, A. Szot, E. Wijmans, V. Koltun, D. Batra, K. Fatahalian. “An Extensible, Data-Oriented Architecture for High-Performance, Many-World Simulation (Madrona).” ACM Transactions on Graphics (SIGGRAPH), 2023.

[2] C. D. Freeman, E. Frey, A. Raichuk, S. Girgin, I. Mordatch, O. Bachem. “Brax — A Differentiable Physics Engine for Large Scale Rigid Body Simulation.” NeurIPS Datasets & Benchmarks, 2021. arXiv:2106.13281.

[3] V. Makoviychuk et al. “Isaac Gym: High Performance GPU-Based Physics Simulation for Robot Learning.” NeurIPS Datasets & Benchmarks, 2021. arXiv:2108.10470.

[4] C. Lu et al. PureJaxRL: end-to-end GPU reinforcement learning in JAX. github.com/luchris429/purejaxrl; see also “Discovered Policy Optimisation,” NeurIPS 2022.

[5] A. Holliday, A. El-Geneidy, G. Dudek. “Learning Heuristics for Transit Network Design and Improvement with Deep Reinforcement Learning.” arXiv:2404.05894, 2024. Code: github.com/AHolliday/transit_learning.

[6] J. Bradbury et al. “JAX: composable transformations of Python+NumPy programs.” 2018. github.com/google/jax.

[7] R. W. Floyd, “Algorithm 97: Shortest Path,” CACM 1962; S. Warshall, “A Theorem on Boolean Matrices,” JACM 1962. (Min-plus / tropical-semiring matrix formulation of APSP.)